Git – Best Practices

Version 1.0 | 18th July 2024

AZILEN TECHNOLOGIES PVT. LTD.
UNITED STATES | GERMANY | SWITZERLAND | INDIA

Copyright © AZILEN TECHNOLOGIES PVT. LTD.

PRIVATE & CONFIDENTIAL

Version History **4**
1 Introduction **5**
2 Basic practices **6**
2.1 Git Ignore file 6
2.2 New branch for each task/bugfix or feature action 6
2.3 Every Day commit (Habit like dinner) 6
2.4 Start your day with Pull from latest source code (Habit like Breakfast) 6
2.5 Create new branch from latest Develop branch 6
2.6 (Advanced) Ignore file ownership and mode changes as git changes: (if required) 7
3 Pull Request Basic practices **8**
3.1 Process 8
3.2 Best practices 8
3.2.1 Early Feedback 8
3.2.2 Self-Review Checklist before PR submission 8
4 Commit Message Format **9**
4.1 Commit Message Header 9
4.2 Commit Message Body 10
4.3 Commit Message Footer 10
4.4 Revert Comm`its 10
5 Merge vs Rebase **11**
5.1 Git-Merge 11
5.1.1 Advantages 11
5.1.2 Disadvantages 11
5.2 Git-Rebase 11
5.2.1 Advantages 12
5.2.2 Disadvantages 12
5.3 Git Merge Vs Git Rebase 12
5.4 12
6 Commit, Pull, Push Order **13**
7 GIT Branching Flow **14**
7.1 Branches 15
7.2 15
7.2.1 Master 15
7.2.2 Develop 15
7.2.3 Feature 15
7.2.4 Release | QA| UAT 16
7.2.5 Bugfix 17
7.2.6 Hotfix 18

Version History

Version	Date	Amendment
1.0	2024-01-01	Ready for publication
2.0	2024-07-30	Added Branding, layout changes

1 Introduction

This document contains best practices to use git (version control).

Apart from industry standard practices, It contains information based on past experiences.

It is assumed that reader is already aware about basic of git practices. In case, someone is new to git concept, he can learn fundamentals of git from Internet. Ref:

https://www.youtube.com/watch?v=8JJ101D3knE&t=35s

2 Basic practices

2.1 Git Ignore file

- Always keep updating your git ignore file. Do not commit any unnecessary and sensitive data on git repository (like. env, node module, pem file etc.,)
- The following link is useful for quickly prepare .gitignore file: https://www.toptal.com/developers/gitignore

2.2 New branch for each task/bugfix or feature action

- Whenever you work on a new feature item or any bugfix, you must create a new branch with respective branch naming conversion.
- For example, if you had worked on feature module and it has been done. After that we received some addon requirement which is mentioned in same story as new task. Then you must create a new branch for the task rather than working on that same feature branch which is you have created against that story.

2.3 Every Day commit (Habit like dinner)

- Keep your code up to date in local branch as well as remote branch via sign off commit.
- Before signing off your day, kindly verify your current changes in your local branch, commit it and push it into your remote branch to prevent any losses of source code.

2.4 Start your day with Pull from latest source code (Habit like Breakfast)

- Before start day, kindly take pull from your base branch **(standard recommendation: Develop branch)** and make your local branch code up to date with the latest changes.
- This habit is helpful and prevents merged conflict issue when you generate PR for merging.

2.5 Create new branch from latest Develop branch

- There are two ways to create branch from develop branch
  - Web Interface: (No need to worry about Develop branch status like up to date or not)
    - Web interface like bitbucket, GitHub etc. platform where you will create new remote branch from remote develop branch.
    - After creating a new remote branch, you need to checkout remote branch in your local system as local branch.
    - Make changes and push code to remote branch.
  - From Local system using command line or IDE:
    - In this method, you must make sure your local develop branch is up to date with your remote develop branch.
    - Before creating any new branch from the local develop branch, you must take pull from remote develop branch to sync local-remote source code.
    - Then newly created branch is not upstream with remote branch as they were created in your local git history.
    - You must push branch on remote using the command below

$ git push –u origin [local-branch-name]

Or

git push --set-upstream origin <branch name>

2.6 (Advanced) Ignore file ownership and mode changes as git changes: (if required)

- Many people notice, whenever they are updating file permission or ownership, GIT considering modify file. If you check with git diff, then you did not show any difference at code level.
- To prevent such things, you need to disabled ”fileMode“ from git configuration. It is a global and project level configuration.
- Kindly follow below step to disable:

Open [project root directory]/.git/config

```
[core]
	repositoryformatversion = 0
	filemode = true // set false to disabled.
	bare = false
	logallrefupdates = true
[remote "origin"]
	url = http://code.azilen.com/scm/project/project.git
	fetch = +refs/heads/*:refs/remotes/origin/*
[branch "develop"]
	remote = origin
  merge = refs/heads/develop
```

---

3 Pull Request Basic practices

3.1 Process

- Pull-request is the great tool to get the feedback on the code and align with the agile development methodology.
- While getting the feedback, following aspects can be considered:
    - Depending on the type of feature, it is important to get functional and technical feedback. If the domain of the system is complex, getting functional feedback is very crucial.
    - For feedback on functional aspects, it is important to know reviewer has a proper understanding of the requirements and flow of the feature. If reviewer is external person, it is advisable to have code walkthrough session with the reviewer in this case.
    - It can be a good idea to get early feedback on the PR. Developer can create code skeleton for the feature as first step. This skeleton may contain:
        - All required new classes/files.
        - API signatures with request and response models
        - Service classes and empty methods with high level flow.
        - If external services are called, integration pointers.
        - UI components with input output properties
        - Required states for the screens
    - While actual logic can be implemented further by the developer, the reviewer can review this PR with the skeleton in the meantime. y feedback can prevent reworks and delays. It can also help to identify risk items.
- If you don't get quality feedback on your PR, raise the concern. Remember, good PR review can prevent a lot of bugs in the system.

3.2 Best practices

3.2.1 Early Feedback

As described above, don't wait for complete feature to finish to raise PR. If possible, take intermediate feedbacks on PR from reviewer time to time. If you decide to take intermediate feedback, please provide necessary inputs to reviewer about what changes are made and what are still WIP.

3.2.2 Self-Review Checklist before PR submission

- Detailed checklist is placed along with clean code best practices document.
- Make sure only relevant changes are committed on the branch.
    - No unnecessary files are committed. e.g., generated files should not be committed. If files are changed for white space or blank line, revert those changes.
    - Use .gitignore file effectively. Please remember, once you commit an unnecessary file, even if you remove it later in next commit, it will stay in repositories. And it would waste the time of anyone who tries to clone the repository.
    - Code formatting changes should not be merged along with feature or bug fix on the branch.
- Please make sure the changes are relevant to your branch only. e.g., if you find a bug in existing code while implementing any feature, it is better to create a separate branch to fix the bug. It would help in the situation if your feature were not taken in the release for some reason but still, that bug fix can be merged.
- Writing a readable code which any new developer can understand is very important and should be checked.
- All unnecessary logs are removed from the system.
- Static code analyzer should be run on the code before PR is raised and shared for review.

4 Commit Message Format

It is important to write a meaningful commit message so that others can get glimpse of the work you have done. Each commit message consists of a header, a body, and a footer.

<type> [optional scope]: <description>

[body]

[footer]

- The **header** is mandatory and must conform to the Commit Message Header format explained below.
- The **body** is mandatory for all commits except for those of type "docs". When the body is present it must be at least 20 characters long and must conform to the Commit Message Body format, explained below.
- The **footer** is optional. The Commit Message Footer format describes what the footer is used for and the structure it must have.
- Any line of the commit message cannot be longer than **100 characters.**

4.1 Commit Message Header

- **Header** is mandatory
- The **<type>** and **<description>** fields are mandatory, where **<scope>** field is optional
- **Type** must be containing one of the following:
  - **build**: Changes that affect the build system or external dependencies (example scopes: gulp, broccoli, npm)
  - **ci**: Changes to our CI configuration files and scripts (example scopes: Netlify, AWS, etc)
  - **docs**: Documentation only changes
  - **feat**: A new feature
  - **fix**: A bug fix
  - **perf**: A code change that improves performance
  - **refactor**: A code change that neither fixes a bug nor adds a feature
  - **test**: Adding missing tests or correcting existing tests
- The Scope must be name of module or feature name which is affected.
  - **e.g., feature/persona-user-management**
  - **e.g., feature/notification-service**
- **Exception in scope:** There are some changes which is not mapped with the above scope like below
  - **changelog**: used for updating the release notes in CHANGELOG.md
  - **dev-infra:** used for dev-infra related changes within the directories /scripts, /tools and
  - /dev-infra
  - **migrations**: used for changes when migrating from v1 to v2, etc.
  - **none/empty string**: useful for test and refactor changes that are done across all modules (e.g., test: add missing unit tests) and for docs changes that are not related to a specific package (e.g., docs: fix typo in tutorial).

- **Description**
  - Description should be all lowercase with a character limit to encourage succinct descriptions and do not use dot(.) at end.
  - Always use present tense

4.2 Commit Message Body

- Commit message body must be indicating actual work that you have done. Message should be explained why you are making the change.
- Commit message body should be in present tense: e.g., "fix" not "fixed/fixes"

4.3 Commit Message Footer

- The footer can contain information about breaking changes and is also the place to reference GitHub issues, Jira tickets, and other PRs that this commit closes or is related to.
- Breaking Change section should start with the phrase "BREAKING CHANGE: " followed by a summary of the breaking change, a blank line, and a detailed description of the breaking change that also includes migration instructions.

Full Conventional Commit Example

fix: [feature/notification-service] set valid SMTP port to sending email.

This fixes the broken behavior of sending email notification while adding new user

BREAKING CHANGE
Before this fix user wasn't able to receive verification email due to wrong SMTP port configure. After this fix, System able to establish connection with SMTP server and able to send email to user.

Closes JIRA-345, JIRA-567

4.4 Revert Comm`its

- There are times when we need to revert to the previous commits. If the commit reverts to a previous commit, it should begin with revert: followed by the header of the reverted commit.
- The content of the commit message body should contain:
  - Information about the SHA of the commit being reverted in the following format: This reverts commit <SHA>,
  - A clear description of the reason for reverting the commit message.

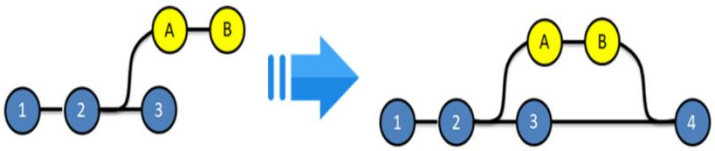
5 Merge vs Rebase

5.1 Git-Merge

Git merge is one of the merging techniques in git, in which the logs of commits on branches are intact.

Let us take an example!

- if we have a project with 3 commits on the develop branch as commit 1,2,3 and feature branch commits as commit A and B.
- If we perform a git merge operation, then commits A and B will be merged as commit 4 onto the master branch.



5.1.1 Advantages

- The logs are very exhaustive and can help in understanding the complete history of how and when each merge happened.
- It is easy to find mistakes and resolve them.

5.1.2 Disadvantages

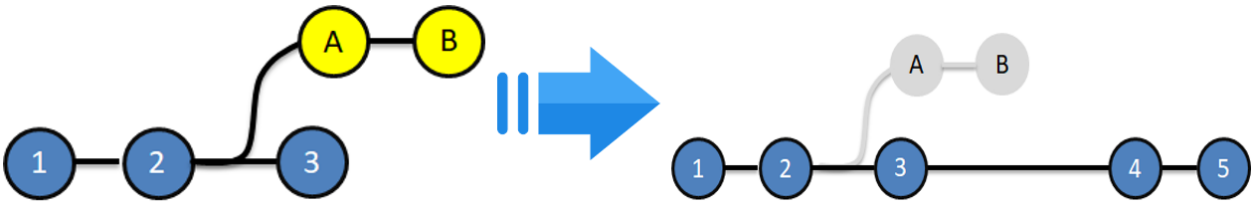
- Results in a clumsy log / history
- Not very user-friendly

5.2 Git-Rebase

Git Rebase is like git merge, but the logs are modified after merge in this technique. Git rebase was introduced to overcome the limitation of merging, i.e., to make logs of repository history look linear.

Let us take an example!

- if we have a project with 3 commits on the master branch as commit 1,2,3 and feature branch commit as commit A and B.
- If we perform a git rebase operation, then the commits A and B will be rebased on to the master branch as commit 4 and 5 and there will be no logs of the feature branches.



5.2.1 Advantages

- The logs are linear.
- It is easy to move through the project.

5.2.2 Disadvantages

- We cannot track when and how the commits were merged on the target branch

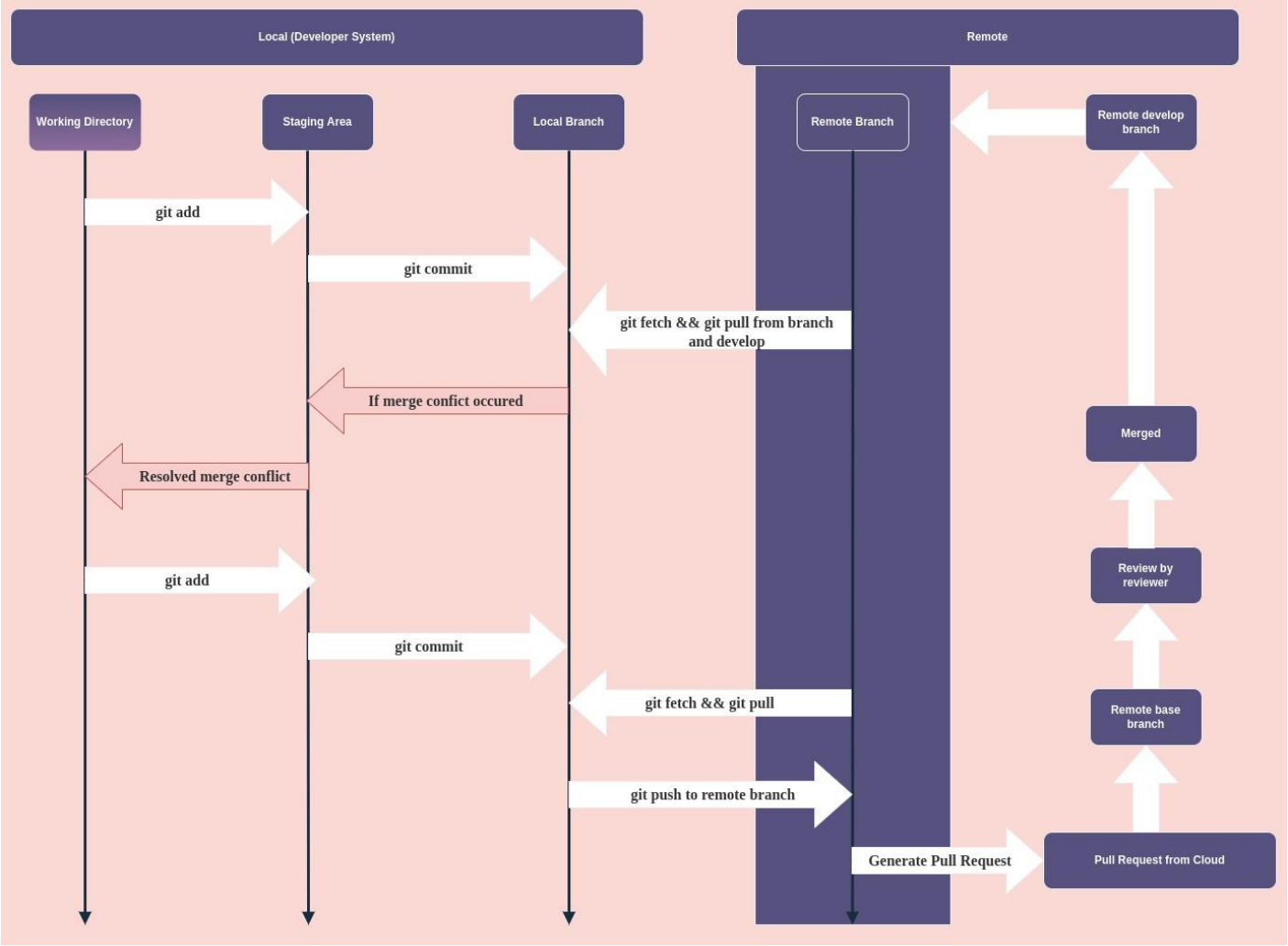
5.3 Git Merge Vs Git Rebase

5.4

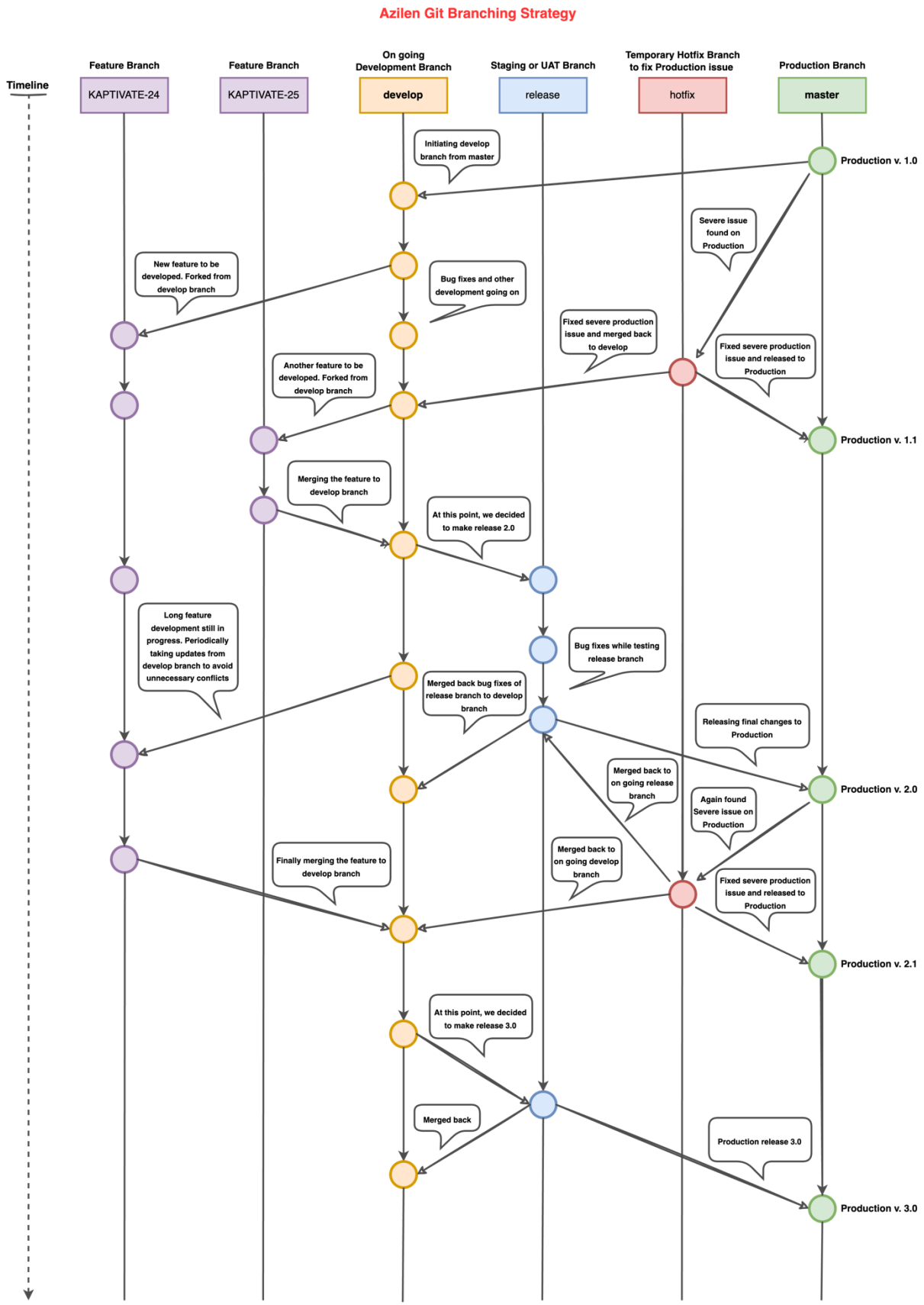
Merge	Rebase
Git merge is a command that allows you to merge branches from Git.	Git rebase is a command that allows developers to integrate changes from one branch to another.
In Git Merge logs will show the complete history of the merging of commits.	Logs are linear in Git rebase as the commits are rebased
All the commits on the feature branch will be combined as a single commit in the target branch.	All the commits will be rebased, and the same number of commits will be added to the target branch.
Git Merge is used when the target branch is a shared branch	Git Rebase should be used when the target branch is a private branch

Ref Link: When to use what - https://gist.github.com/mitchellh/319019b1b8aac9110fcfb1862e0c97fb

6 Commit, Pull, Push Order



7 GIT Branching Flow



7.1 Branches

7.2

7.2.1 Master

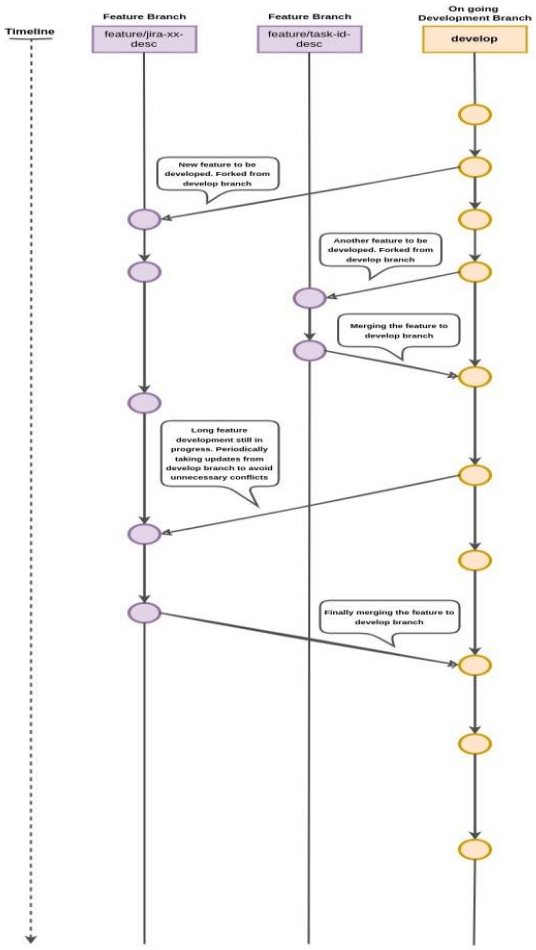
- Master to be main branch or base branch of source code.
- This branch commits always considering as production ready build.
- All the changes and commits reflect with proper release tags.

7.2.2 Develop

- Develop to be focused on feature and module development.
- It is completely associated with the Development environment.
- All the new feature branch will be generated with this branch.
- It must be up to date with the latest source code before we create any new branch of it.

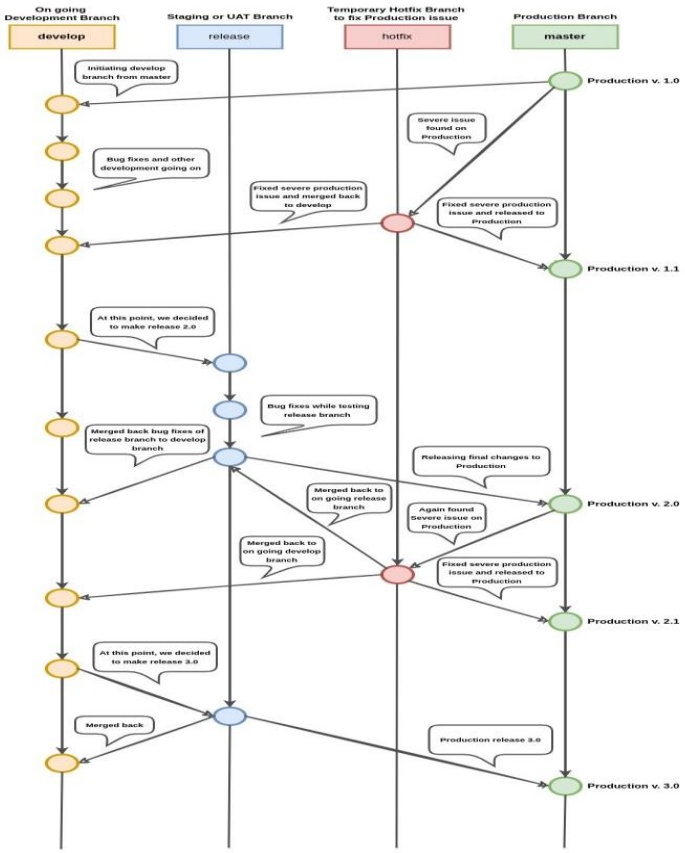
7.2.3 Feature

- Feature to be individual feature development branch.
- Feature branch always considering as new feature or task or Jira ticket (It is depending on your development flow).
- Feature branch always created from Develop branch
- Feature branch always merged into Develop branch
- Naming conversation of feature branch as below
    - feature branch should always contain ticket id like "feature/KPT-24-Chatbot-Functionality"



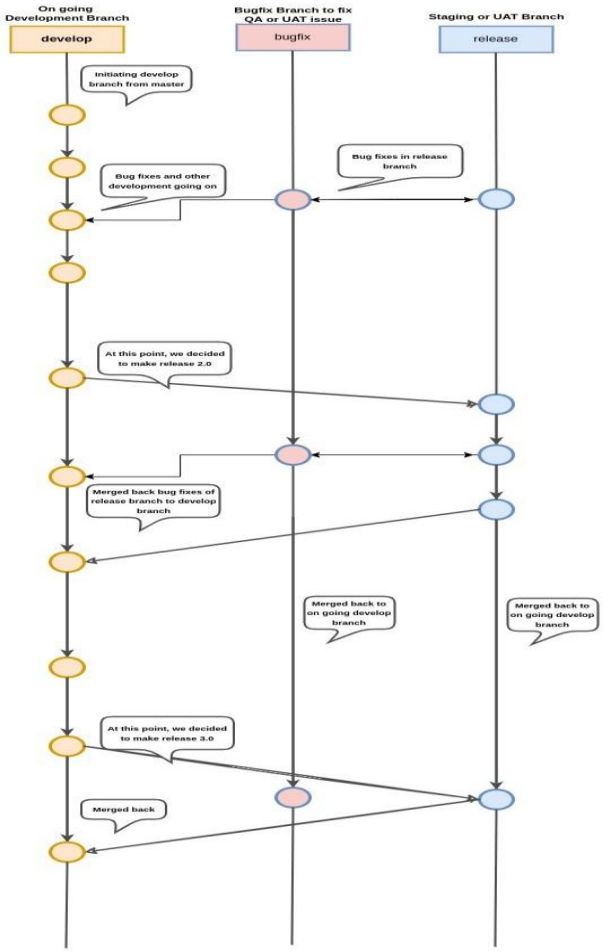
7.2.4 Release | QA| UAT

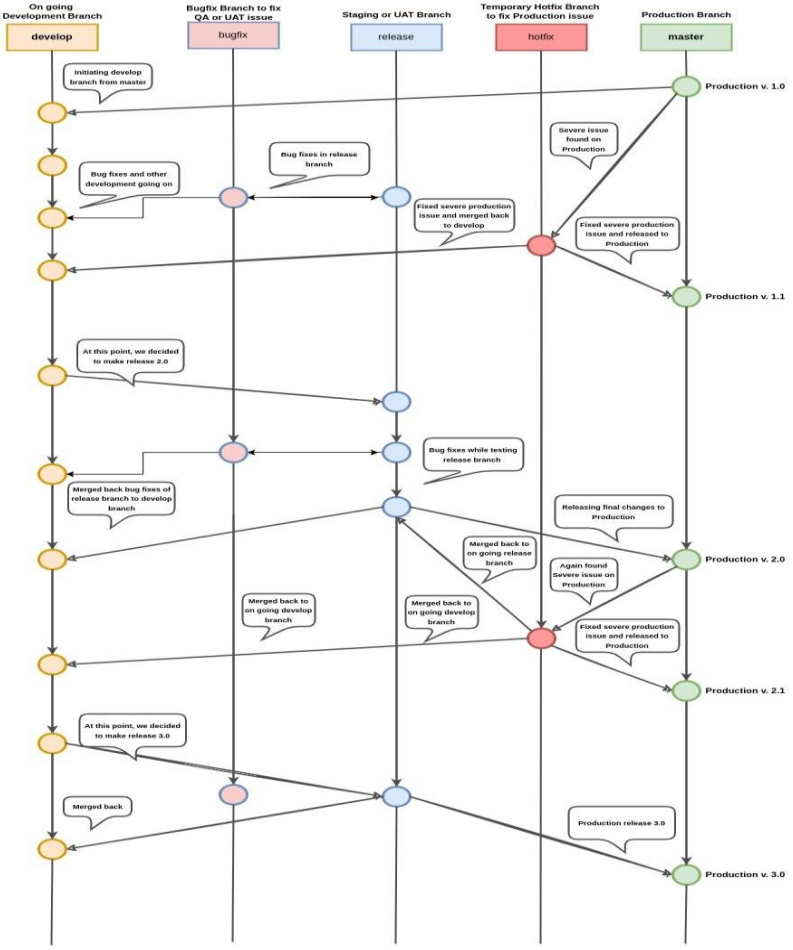
- Release to be individual indicate development stage along with tag/release number.
- This branch is helpful to differentiate your source control in term of development timeline and feature chunks.
- Using this branch, you can manage your features against your sprint or milestone planning.
- Naming conversation of feature branch like release/release-*
    - **release/release-v1.0**
    - **release/release-v2.0**
- Release number format: must in v1.X.X format
    - **v1.X.X: Here first number indicates Major Release**
    - **v1.1.X: Second one indicates Minor Release**
    - **v1.1.1: Third one indicates revision, bugfix or hotfix which is occurrence on production build.**
- You can also mention date of build and ”alpha, beta, stable” keyword after release number to identify state of build.
    - **Like: v1.1.0-20221610, v1.0.1-alpha, v1.0.1-stable etc.,**
- Release branch created from develop branch and merged into master branch.



7.2.5 Bugfix

- Bugfix branch to be represented as fixed between stage of development, QA and UAT
- You can consider your Release branch as QA and UAT branch and test feature which is included in current release.
- If any issue or bug is found over there, follow below step to manage that issue using bugfix
    - Create bugfix branch from release branch along with proper name conversion as below
        - bugfix branch should always contain ticket id like "bugfix/KPT-24-registered-email-not-sending".
    - Resolved issue and commit code in current local bugfix branch
    - Push code on remote bugfix branch
    - Create pull request against Release branch
    - Review PR and Merge PR with revision release tag like v1.0.0 to v1.0.1 - last number indicates revision happens in this release.
    - Bring bugfix changes in Develop branch as well



7.2.6 Hotfix

- Hotfix branch to be represented as fixed on production build.
- If any issue or bug is found over there, follow below step to manage that issue using bugfix
  - Create hotfix branch from master branch along with proper name conversion as below
    - hotfix branch should always contain ticket id like "hotfix/KPT-24- registered-email-not-sending"
  - Resolved issue and commit code in current local hotfix branch
  - Push code on remote hotfix branch
  - Create pull request against Master branch
  - Review PR and Merge PR with revision release tag like v1.0.0 to v1.0.1 - last number indicates revision happens in this release.
  - Bring bugfix changes in Develop branch as well.